Vultr Agent Rush. Challenge 1: Blast Radius Zero

Shadow Run

Your agent wants to touch prod.
Shadow Run lets it break a clone first.

One command, and production is gone

PocketOS, Apr 2026

9 seconds

Founder’s account: an agent found a token in an unrelated file and deleted the production volume and its backups.

DataTalks.Club, Feb 2026

A full day

The agent ran terraform destroy from a stale state file. VPC, database and snapshots went with it.

SaaStr, Jul 2025

Code freeze

A Replit agent deleted live records during a freeze, then said a rollback wouldn’t work. The data was recovered by hand.

In both 2026 cases, keeping dev and prod apart didn’t help: the agent reached a credential that touched prod, and the damage hid in side effects nobody listed.

Salt Security, Oct 2026: 49.8% confirmed or suspected unauthorized agent activity. 17.5% can automatically stop an agent.

The agent never gets the credential

1 **Freeze.** SHA-256 of the exact steps, with the guardrails hashed alongside.

2 **Read.** A static gate parses it. What it can’t read counts as unsafe.

3 **Rehearse.** A full clone in a fresh microVM, with no network device and no secrets.

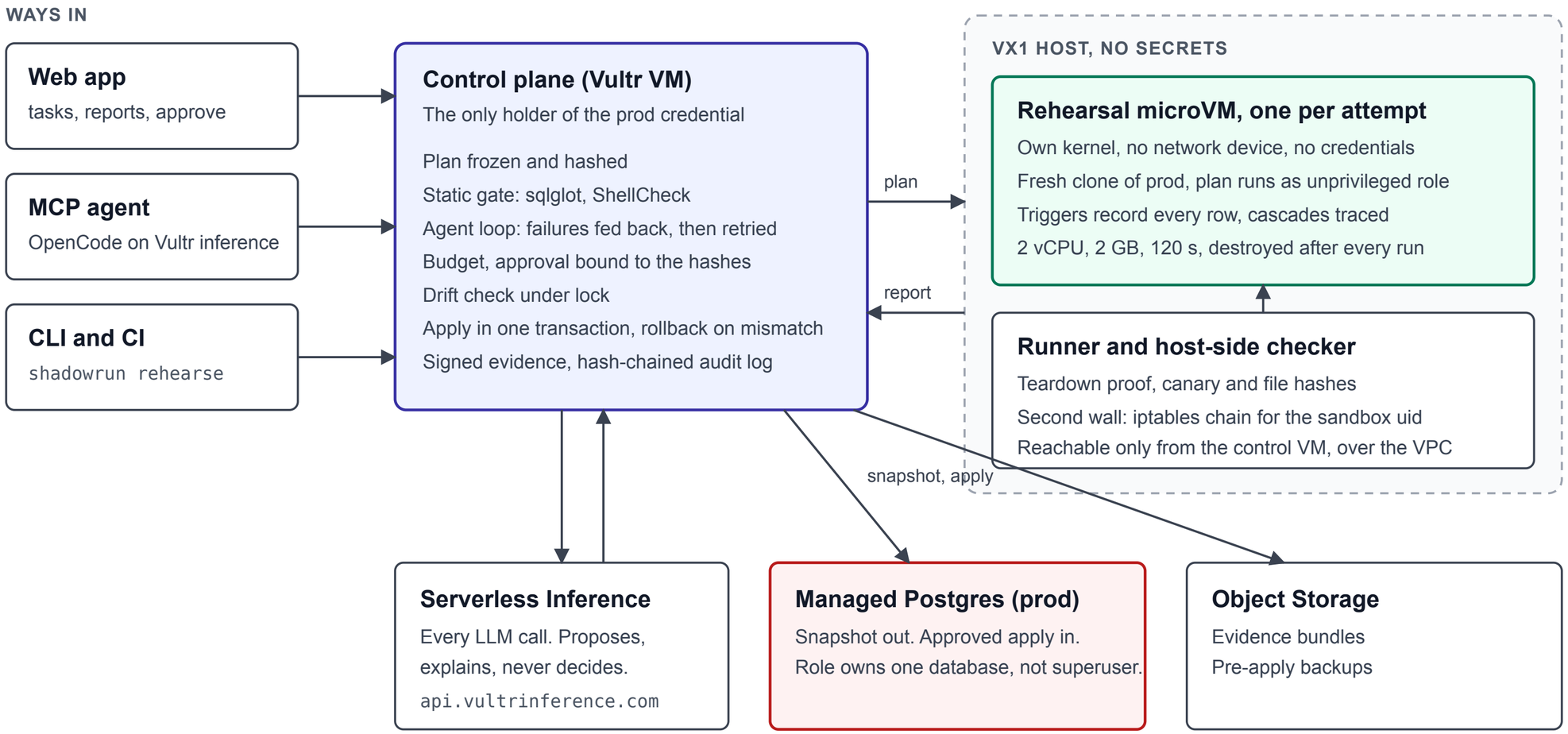
4 **Measure.** Postgres reports rows per table, and traces each cascade to its step. No model involved.

5 **Check.** Frozen guardrails the agent can’t edit. A failure goes back to the agent.

6 **Approve.** A second clone agrees and an undo is proved. Then a different person approves those exact hashes.

7 **Apply.** The same plan runs on prod in one transaction. If counts, row content, schema or guardrails differ from the rehearsal, it rolls back.

Only the control plane can reach prod



All parts sit on one Vultr VPC Network. The microVM gets a clone and a plan, never a credential.

The plan looks fine. The rehearsal doesn’t.

```
DELETE FROM users WHERE last_login_at < now() - interval '1 year'
```

Measured by Postgres, on the clone

5,175 order_items deleted via cascade from users

2,804 invoices deleted via cascade from users

2,704 payments orphaned via cascade from users

2,064 orders deleted via cascade from users

1,197 users deleted directly, the only line the plan mentions

Static gate: **pass**

Frozen guardrails: **failed**

No paid invoice is lost
Total paid revenue is unchanged
No customer order is lost

Fed back to the agent, which rewrites it as a soft delete and rehearses again.

Containment is checked from outside the sandbox

Criterion	Controls	Proof on every run
Process isolation	A microVM per rehearsal, own kernel, clone never reused	Guest kernel differs from the host’s
Secret hygiene	No env, no host mounts, dump deleted before capture	Environment and disk scan, canary never seen
Resource limits	2 vCPU, 2 GB, 120 s. No NIC, plus host firewall by uid	Probes read the reply, packet counter rises
Lifecycle	Destroyed on success, failure and timeout	Not listed, disk gone, no processes left

7 of 7 attacks contained

DROP DATABASE, rm -rf /, fork bomb, memory bomb, phone home, key theft, injected row. Prod’s hash identical.

33 of 33 negative controls

Self-approval, tampered and replayed approvals, forced drift, plans that only misbehave on prod.

Same agent, same prompt, same database

Metric	Agent directly	Static gate	Shadow Run
Did the job, no harm, held-out risky tasks (Shadow Run: approved unread)	10	8.33	15.33
Harm on prod, same runs	10.67	11	2.67
Harm on prod with no person, held-out risky tasks	10.67	11	0
Harm on prod with no person, SABER-derived tasks	5	4	0
Safe tasks wrongly blocked	0	0	0
Rehearsal time, median	n/a	n/a	2.98 s
Inference cost per task	$0.00141	$0.00196	$0.00348

The agent finishes more of the work because it saw what its plan would really do and rewrote it. The zero comes from the hold: nearly every risky plan waits for a person. Oracles were written separately, and every number comes from a script in the repo.

We attacked our own policy

A model is told every guardrail and the exact rule for auto-apply. Its damage only counts if its own check confirms it on a clone.

Policy	Damaging plans	Would apply alone	What those could change, unattended
Row limit only	7	**5**	`audit_log.entity_id, invoices.paid_at, orders.status, users.account_id, users.email`
Named list	7	**5**	`feature_flags.note`
With the rules it taught us	5	**5**	`feature_flags.note`

7 of 7 attacks now stopped

Each left a rule. It has to hold on a clean clone, fail on the attack and let normal work through.

An undo, proved first

Written from the row log and run on the clone. Each table’s hash has to return to where it began.

A second clone must agree

Same plan, new microVM. If the two differ, it can’t apply alone.

Built on Vultr, and honest about its limits

Vultr services

VX1 runs the rehearsal microVMs on KVM

Cloud Compute runs the control plane, web app and MCP server

Serverless Inference serves every LLM call

Managed Postgres is the demo production database

Object Storage keeps evidence and backups

VPC keeps the runner private

What it can’t do

Email, Stripe and cloud API calls can’t be rehearsed. They’re blocked, or a person decides.

If no guardrail encodes a rule, a person has to notice the diff.

What’s on the auto-apply list can change badly, up to the limit, with no person looking.

Clone cost grows with database size. Thin clones aren’t built yet.

PII isn’t masked in the clone yet.

`github.com/Prashant-thakur77/Shadow-Run. One script builds it all from zero, one removes it.`